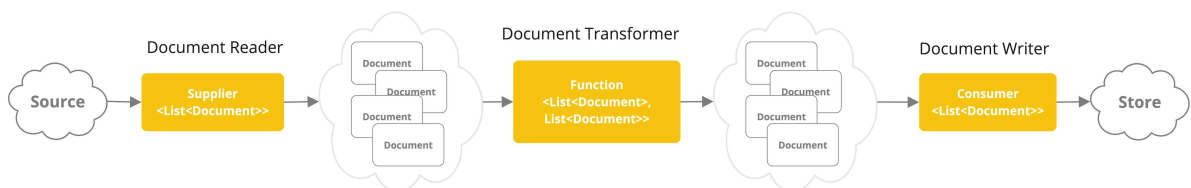




고품질 RAG를 위한 데이터 전처리 - Extract, Transform, Load

1. ETL 파이프라인이란?

- RAG(검색 증강 생성) 시스템의 성능은 결국 'Vector DB에 얼마나 깨끗하고 잘 정돈된 데이터가 들어있는가'에 달려 있음
- 아무리 뛰어난 LLM과 고급 Advisor를 사용하더라도, 원본 데이터가 엉망이면 AI는 엉뚱한 대답을 내놓게 됨
- Spring AI는 다양한 형태의 원본 데이터를 AI가 소화하기 가장 좋은 형태로 가공하여 DB에 밀어 넣는 **ETL(Extract, Transform, Load)** 파이프라인을 체계적인 클래스 구조로 지원함



ETL Pipeline :: Spring AI Reference

The Extract, Transform, and Load (ETL) framework serves as the backbone of data processing within the Retrieval Augmented Generation (RAG) use case.

 <https://docs.spring.io/spring-ai/reference/api/etl-pipeline.html#page-title>

2. 1단계: 추출 (Extract) - Document Reader

- 다양한 데이터 소스(파일, DB, 웹페이지 등)에서 텍스트를 읽어와 Spring AI의 기본 규격인 **Document** 객체 리스트로 변환하는 핵심 컴포넌트
- 텍스트 및 웹 데이터 추출기

- **JsonReader** : JSON 파일에서 특정 키(Key)의 값만 쓱쓱 뽑아냄.
- **TextReader** : 단순 텍스트 파일을 하나의 **Document** 로 변환
- **JsoupDocumentReader** : 크롤링할 때 많이 쓰는 Jsoup을 기반으로, HTML 문서에서 특정 CSS 선택자(Selector)에 해당하는 부분만 정밀하게 추출
- **MarkdownDocumentReader** : 마크다운의 구조(수평선, 코드 블록 등)를 이해하고, 그 기준에 맞춰 문서를 지능적으로 분할해 줌
- PDF 및 오피스 문서 추출기
 - **PagePdfDocumentReader** : PDF 문서를 '페이지 단위'로 잘라서 여러 **Document** 로 만듦
 - **ParagraphPdfDocumentReader** : PDF 내부에 있는 목차(TOC) 정보를 이용해 '논리적인 단락별'로 잘라줌
 - **TikaDocumentReader** : Apache Tika 엔진을 사용하여 PDF, DOC/DOCX, PPT/PPTX, HTML 등 수많은 포맷을 이 리더 하나로 전부 처리할 수 있는 강력한 클래스

3. 2단계: 변환 (Transform) - **DocumentTransformer**

- 추출된 거대한 **Document** 를 AI 모델의 제한된 기억력(Context Window)에 맞게 잘게 쪼개거나(Chunking), 검색이 더 잘 되도록 유용한 정보를 붙이는 가공 단계
- 문서 분할 (Chunking)
 - AI가 한 번에 읽을 수 있는 토큰 수에는 한계가 있기 때문에, 긴 문서를 적절한 크기로 자르는 것은 매우 중요함
 - **TokenTextSplitter** 등을 활용하여 분할함
- 메타데이터 자동 생성 및 강화
 - 데이터를 가공하는 과정에서 **ChatModel(AI)**을 직접 호출하여 문서를 더 똑똑하게 태깅 할 수 있음
 - **KeywordMetadataEnricher** : AI에게 문서를 읽고 **핵심 키워드**를 뽑아내게 함
 - **SummaryMetadataEnricher** : AI에게 문서 내용의 **요약본**을 작성하게함

4. 3단계: 적재 (Load) - **DocumentWriter**

- 모든 가공이 끝난 **Document** 객체 리스트를 최종 목적지에 저장하는 단계
- VectorStore : 앞서 우리가 배웠던 Vector DB 구현체들은 모두 기본적으로 이 **DocumentWriter** 인터페이스를 구현하고 있음

```
public interface VectorStore extends DocumentWriter, VectorStoreRetriever {  
  
    default String getName() {  
        return this.getClass().getSimpleName();  
    }  
  
    void add(List<Document> documents);  
  
    void delete(List<String> idList);  
  
    void delete(Filter.Expression filterExpression);  
  
    default void delete(String filterExpression) { ... }  
  
    default <T> Optional<T> getNativeClient() {  
        return Optional.empty();  
    }  
}
```

- 즉, `vectorStore.add(documents)` 를 호출하는 순간 적재가 됨
- FileDocumentWriter : 가공된 문서 리스트를 사람이 눈으로 읽기 쉬운 텍스트 파일로 뱉어냄. 주로 디버깅 할 때 사용